

Prompt-

you are cofounder and senior software architect and you are building one saas product named as Jarvis CRM- this is not typical CRM its having lead intelligence radar and internal ai based tracking system.

problem we are going to solve with this is -

1. companies having various types of data which they want to make it uniform for marketing and outbound campaigning
2. crucial data they can not hand over to telecallers or sales person as it might stolen for which company has paid
3. no control over database access and no monitoring on data processing and lead tracking.calling mailing messaging and conversion pipeline stages
4. for lead generation company needs to scrap it from various sites and need to pay huge for such data

our software will be one stop solution for all - it will be CRM with global data access which they can subscribe to get in pad plan.

saas admin and his team will keep uploading global data, which the paid user can see and if intrested can pull in his account. then user can allocate that data to his telecaller or team and track conversion stages and other CRM records.

here system will be like this-

super admin can create universal global database by uploading various kinds of excel files-

there will be two main database will be created first is company and other is people registry. then relation will be mapped. for example a PVT ltd company will be having unique CIN with various info such as company name, address, CIN, city, website, email, mobile number and so on. same company will be having 2 directors. so that director database will be maintained in different people registry database as 1 direct May have more than 1 company under his name. then people personal database will be maintained as DIN, name , email,mobile, LinkedIn, other accounts, company regisitered under his name and so on.

the admin user can create subusers, who will upload database using excel format , and universal data is updated. there will be upload date and time will be maintained and last update record if database is updated.

platform paid user will have admin access where he will setup his company and can create subusers who will have limited access to CRM. he will have his own database where he can also upload excel csv or using input form he can insert database. he will have company name as lead and inside that people related to that company.

it will be pwa friendly where user can call with one click, can Whatsapp with one click and can email as well. admin can customize lead or sales pipeline or stages. and user can have record dof each activity like he called then call logs, message logs or followup then followup calendar will be updated and auto reminder will be set. which user can edit anytime.

admin will have comprehensive dashboard. also in global database, he can browse from various filters and add that database to his account.

than that data he can assign to his sub user which sub user can see and work on it.

all CRM features will be there and in future we will have ai integration so have provision of that also Mcp friendly.

also while creating get auto debugging or tracing making vibe coding easy and testing easy.

maintain documentation with source of truth and Bible and other development documents.

this web app will be deployed on vps mainly hostinger so make it deployed friendly and with ci cd pipeline for GitHub push commit and so on making it easy to update using GitHub.

in CRM if user wants to upload docs or images or any file then that will be maintained on separate database which in future can migrate to other server in one click using API access.

get me complete srs and prd to develop this using ai vibe coding.

Chat gpt response-

Yes. I would architect **Jarvis CRM as a data-intelligence and controlled-outbound platform**, not as another Salesforce/Zoho-style CRM.

The most important architectural decision is this:

Global Data ≠ Customer Data ≠ CRM Activity Data ≠ File Storage.

They should be separate domains with strict ownership, permissions, audit trails and controlled promotion/copying between them.

Below is the **master PRD + SRS + technical architecture + AI-vibe-coding development specification** I would use as the source of truth for building Jarvis.

JARVIS CRM

Lead Intelligence Radar + Controlled Outbound CRM

Document Type: Master Product Requirements + Software Requirements Specification

Product: Jarvis CRM

Version: 1.0

Status: Product/Architecture Source of Truth

Target: Multi-tenant SaaS / PWA / VPS deployment

Primary Deployment: Hostinger VPS + Docker + GitHub CI/CD

1. PRODUCT VISION

Jarvis CRM is not a traditional CRM.

It is a:

Lead Intelligence + Global Business Database + Controlled Sales Execution + AI-ready CRM platform.

Jarvis solves four fundamental business problems:

Problem 1 — Fragmented Data

Companies receive data from:

- Excel
- CSV
- websites
- directories
- purchased databases
- internal databases
- lead-generation agencies
- trade portals
- government/company registries
- marketing campaigns
- manually collected contacts

The data has different formats and structures.

Jarvis converts heterogeneous data into a standardized:

Company Registry + People Registry + Relationship Graph

Problem 2 — Sensitive Data Exposure

Companies may spend substantial money acquiring business intelligence.

They cannot safely provide the entire database to:

- telecallers
- sales executives
- interns
- external agencies
- temporary employees

Jarvis therefore introduces:

Controlled Data Access

Users receive only the records they are authorized to work on.

Problem 3 — No Visibility Into Data Usage

Management often cannot answer:

- Who accessed this lead?
- Who downloaded it?
- Who called it?
- Who changed it?
- Who exported it?
- Who sent WhatsApp?
- Who emailed?
- Who assigned it?
- How many times was it contacted?
- What happened after contact?
- Why did the lead become lost?
- Which salesperson is actually working?
- Which database source generates conversions?

Jarvis provides:

Complete Lead Activity + Data Access Auditability

Problem 4 — Expensive Lead Generation

Instead of every company independently purchasing/scraping data, Jarvis maintains a:

Global Intelligence Database

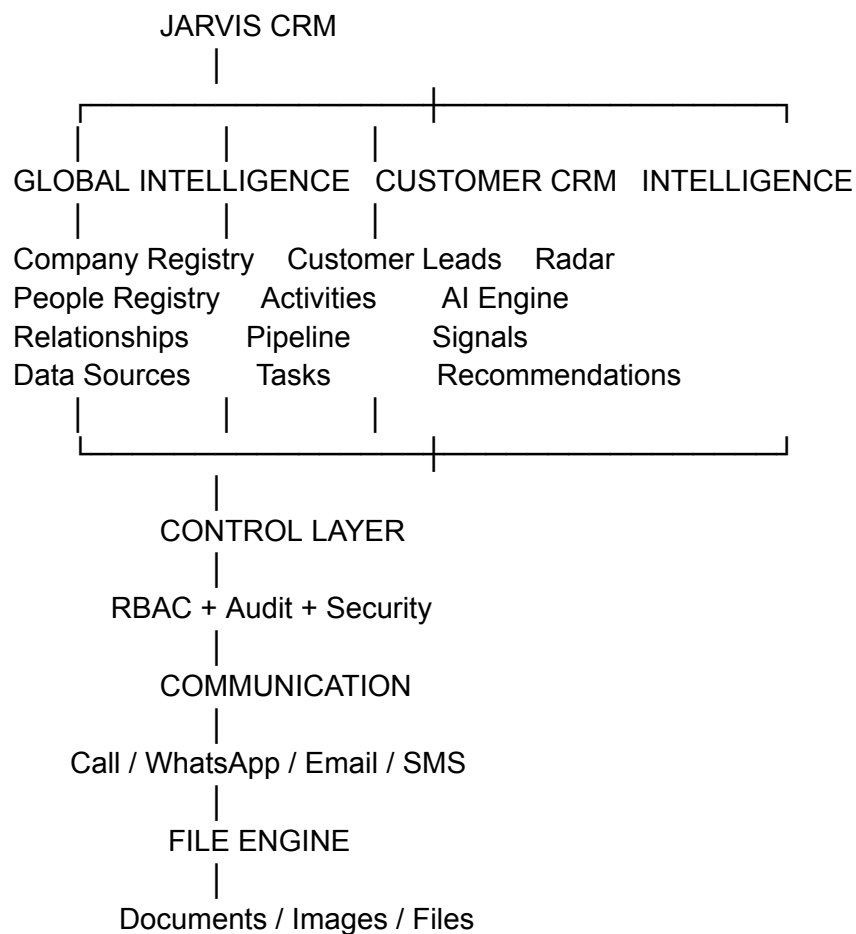
The SaaS platform administrator/team can upload and maintain business intelligence.

Subscribers can:

1. Search global data
 2. Filter it
 3. View permitted fields
 4. Select records
 5. Import/pull records into their private workspace
 6. Assign those records to employees
 7. Execute campaigns
 8. Track conversion
-

2. CORE PRODUCT CONCEPT

Jarvis consists of five major layers.



3. PRODUCT TERMINOLOGY

The following terminology becomes part of the Jarvis product language.

Global Database

Data owned/managed by Jarvis platform.

Customer Database

Private database belonging to a subscribing organization.

Company Registry

Canonical database of organizations/business entities.

People Registry

Canonical database of people associated with companies.

Relationship

Connection between people and organizations.

Examples:

- Director
- Founder
- CEO
- CFO
- Procurement Head
- HR Head
- Employee
- Consultant
- Partner

Lead

A customer-owned prospect being actively worked by the organization.

Global Record

Record existing inside the Jarvis global intelligence database.

Pulled Record

Global record copied/referenced into a customer's workspace.

Assignment

Allocation of a lead to a user/team.

Activity

Any interaction or system event involving a lead.

Signal

An event indicating potential buying intent or business opportunity.

Radar

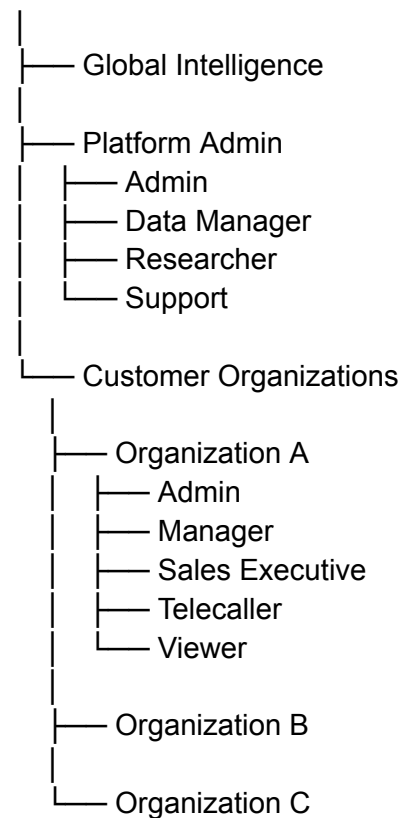
Jarvis intelligence layer that detects and prioritizes opportunities.

4. MULTI-TENANT MODEL

Jarvis must be multi-tenant from Day 1.

Hierarchy:

Jarvis Platform



Every customer-owned record must contain:

tenant_id
created_by
updated_by
created_at
updated_at

No customer query should be capable of accessing another tenant's records.

5. THREE-DATABASE DOMAIN MODEL

Do NOT create one giant CRM table.

Jarvis should conceptually separate:

A. Global Intelligence Domain

global_companies
global_people
global_company_people
global_company_addresses
global_company_contacts
global_company_identifiers
global_sources
global_data_updates
global_import_batches
global_data_quality

B. Customer CRM Domain

crm_companies
crm_people
crm_leads
crm_assignments
crm_activities
crm_calls
crm_messages
crm_emails
crm_followups
crm_tasks
crm_pipeline_stages
crm_pipeline_history
crm_notes
crm_tags

crm_campaigns

C. Platform/System Domain

organizations
users
roles
permissions
subscriptions
plans
usage
audit_logs
login_events
api_keys
webhooks
notifications
system_events
feature_flags

6. COMPANY REGISTRY

The Company Registry is the canonical business entity system.

Possible fields:

company_id
legal_name
display_name
company_type
registration_number
CIN
tax_identifier
GSTIN
country
state
city
address
postal_code
website
domain
primary_email
primary_phone
secondary_phone
industry
sub_industry

employee_range
revenue_range
founded_year
status
source
source_record_id
data_confidence
last_verified_at
created_at
updated_at

The schema must remain configurable.

Do not hard-code every possible global data provider's fields into the core model.

Use an extensible attribute system where appropriate.

7. PEOPLE REGISTRY

People are independent entities.

A person may belong to multiple companies.

Fields:

person_id
DIN
full_name
first_name
middle_name
last_name
designation
email
mobile
alternate_mobile
linkedin_url
website
country
city
professional_summary
source
source_record_id
data_confidence
last_verified_at
created_at

updated_at

8. COMPANY ↔ PERSON RELATIONSHIP

Never store directors only as columns inside company records.

Create relationship records.

Example:

company_people

id
company_id
person_id
relationship_type
designation
start_date
end_date
is_current
source
created_at
updated_at

Example:

ABC Pvt Ltd

- |
- | — Person A → Director
- | — Person B → Director
- | — Person C → CFO
- | — Person D → Procurement Head

And:

Person A

- |
- | — ABC Pvt Ltd
- | — XYZ Ltd
- | — PQR Industries

This creates the foundation for Jarvis Intelligence.

9. GLOBAL DATA INGESTION

Platform administrators can upload:

- XLSX
- CSV
- JSON
- API feeds
- future connectors

Upload workflow:

Upload File



File Validation



Column Detection



Schema Mapping



Data Preview



Duplicate Detection



Entity Resolution



Relationship Resolution



Data Quality Validation



Import



Audit



Indexing

10. SMART COLUMN MAPPING

Jarvis should recognize variations such as:

Company Name

company

Organization

Legal Name

Business Name

Firm Name

and map them to:

legal_name / display_name

Similarly:

Mobile

Mobile Number

Phone

Contact

Telephone

→

phone

The mapping engine should provide:

Automatic Mapping

Jarvis suggests mappings.

Manual Mapping

Admin can correct them.

Saved Mapping Template

Example:

Indian MCA Export Template

Africa Buyer Template

LinkedIn Export Template

Trade Directory Template

Internal CRM Template

11. DATA NORMALIZATION

Before insertion:

Phone

Normalize:

+91 98765 43210

09876543210
9876543210

into canonical format.

Email

Normalize:

- lowercase
- trim
- validate

Website

Normalize:

http://example.com
https://www.example.com/
example.com

to canonical domain representation.

Company

Normalize:

ABC Pvt Ltd
ABC PRIVATE LIMITED
ABC PVT. LTD.

into an entity-resolution candidate.

12. DUPLICATE DETECTION

Duplicate priority:

Company

1. CIN
2. Registration number
3. Tax ID
4. Domain
5. Exact normalized company name + location
6. Fuzzy company name

Person

1. DIN
2. Email
3. Mobile
4. LinkedIn URL
5. Name + company relationship

The system should never blindly merge uncertain entities.

Use:

CONFIRMED
PROBABLE
POSSIBLE
CONFLICT

13. DATA PROVENANCE

Every global record must maintain provenance.

Example:

Source:
MCA Dataset

Imported:
2026-09-07 14:35

Batch:
IMP-20260907-004

Source Record:
MCA-123456

Last Updated:
2026-09-07

Confidence:
92%

This is extremely important for enterprise trust.

14. GLOBAL DATA BROWSER

Platform admin gets a powerful data explorer.

Features:

- global search
- company search
- people search
- industry
- country
- state
- city
- company size
- designation
- source
- data freshness
- confidence
- registration type
- relationship
- tags
- upload batch
- last updated
- missing fields

Example:

Country: Kenya

Industry: Energy

Company Type: Private

Person Designation: Procurement

Has Email: Yes

Has Phone: Yes

Has Website: Yes

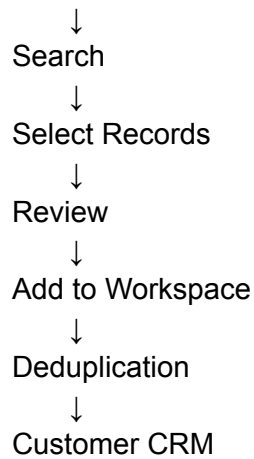
Data Freshness: < 180 days

15. GLOBAL DATA → CUSTOMER DATABASE

The user should NOT simply "download Excel."

Instead:

Global Database



System records:

global_record_id
customer_id
imported_by
imported_at
source

This provides traceability.

16. GLOBAL DATA LICENSING / CONTROL

The platform must support configurable rules for global data.

Examples:

Can View
Can Pull
Can Export
Can Reveal Email
Can Reveal Phone
Can Assign
Can Download

These should be controlled by:

- subscription plan
- organization
- user role
- purchased credits

- data licensing rules

This is critical because some datasets may have contractual restrictions.

17. CUSTOMER ORGANIZATION

When a company subscribes:

Create Organization



Organization Admin



Company Profile



Users



Roles



Teams



Pipelines



CRM Settings

Organization profile:

organization_id

company_name

legal_name

industry

website

country

address

logo

timezone

currency

subscription_plan

subscription_status

18. CUSTOMER USERS

Admin can create:

Admin
Manager
Sales Executive
Telecaller
Researcher
Marketing
Viewer
Custom Role

Roles must be configurable.

19. RBAC

Permission model:

MODULE
RESOURCE
ACTION
SCOPE

Example:

Lead
View
Assigned Only

Another:

Lead
View
Team

Another:

Lead
Export
None

Permissions:

create
read
update
delete
assign
export

import
call
message
email
view_sensitive_data
download
bulk_action
manage_pipeline
manage_users
manage_settings

20. DATA ACCESS CONTROL

A telecaller should never automatically receive the entire customer database.

Example:

Customer Admin
↓
Manager
↓
Team
↓
Telecaller
↓
Assigned Leads

The telecaller sees:

Assigned Lead
Company
Allowed Contact
Allowed Phone
Allowed Email
Activities
Followups

but not:

entire database
other team's leads
global raw dataset
customer acquisition source
restricted fields

21. FIELD LEVEL SECURITY

Some fields may be restricted.

Example:

Person Name → visible
Company → visible
Mobile → visible
Personal Email → restricted
Data Source → admin only
Acquisition Cost → admin only
Internal Score → manager

Therefore permissions should eventually operate at:

Record Level

+

Field Level

+

Action Level

22. CRM LEAD MODEL

A lead may represent:

Company

+

Contact Person

+

Opportunity

Lead structure:

lead_id
organization_id
company_id
primary_person_id
source
status
pipeline_id
stage_id
owner_id
team_id
priority

lead_score
temperature
expected_value
expected_close_date
tags
created_at
updated_at

23. PIPELINE ENGINE

Pipeline must be configurable.

Example:

New
↓
Contacted
↓
Connected
↓
Interested
↓
Meeting Scheduled
↓
Proposal
↓
Negotiation
↓
Won

Another organization may use:

New
Qualification
Demo
Trial
Proposal
Payment Pending
Won
Lost

Never hard-code these stages.

24. PIPELINE HISTORY

Every movement must be recorded.

lead_id
from_stage
to_stage
changed_by
changed_at
reason
duration_in_previous_stage

This enables:

- conversion analytics
 - bottleneck detection
 - salesperson performance
 - AI predictions
-

25. ACTIVITY ENGINE

Everything should become an activity.

Activity types:

CALL
WHATSAPP
EMAIL
SMS
NOTE
FOLLOWUP
MEETING
TASK
STAGE_CHANGE
ASSIGNMENT
STATUS_CHANGE
DOCUMENT
COMMENT

Activity model:

activity_id
lead_id
user_id
type

direction
status
subject
description
timestamp
duration
metadata
created_at

26. CALL ENGINE

PWA-friendly:

Call

button.

Browser/mobile launches phone handler where supported.

System records:

call initiated
call timestamp
user
lead
number
duration if provider supports it
outcome
notes
next followup

Important:

The browser cannot guarantee access to native call duration or call recording on every device.

Therefore the architecture should support future telephony integrations.

27. WHATSAPP

One-click WhatsApp should use approved WhatsApp mechanisms.

Architecture:

Jarvis



WhatsApp Connector



WhatsApp Business Platform / provider

Track:

message

template

recipient

sender

timestamp

delivery

read

reply

campaign

lead

Do not design the core system around unofficial WhatsApp automation.

28. EMAIL

Support:

SMTP

Transactional Email Provider

Future Gmail/Microsoft connectors

Track:

sent

delivered

bounced

opened

clicked

replied

Where provider support exists.

29. FOLLOW-UP ENGINE

When a user creates:

Follow-up:

Tomorrow 11:00 AM

system creates:

followup

calendar_event

notification

Reminder:

1 day before

1 hour before

at due time

These must be configurable.

30. TASK ENGINE

Tasks can be:

Call client

Send quotation

Follow up

Schedule meeting

Verify company

Research contact

Send proposal

Task:

task_id

assigned_to

created_by

lead_id

priority

due_date

status

completion_date

31. LEAD INTELLIGENCE RADAR

This is the feature that differentiates Jarvis.

Traditional CRM:

User enters lead
→ User follows lead

Jarvis:

Global Data
+
CRM Data
+
Activities
+
Signals
+
External Intelligence
↓
JARVIS RADAR
↓
Opportunity Detection
↓
Priority
↓
Recommended Action

32. RADAR CONCEPT

Radar identifies:

New Opportunity

A company matching the user's ICP.

Buying Signal

Potential indication that the company may need the user's product/service.

Contact Change

Relevant decision-maker changed.

Company Change

New:

- plant
- branch
- project
- expansion
- funding
- hiring
- tender
- website activity

Stale Lead

Lead hasn't been contacted.

Follow-up Risk

Follow-up overdue.

Conversion Risk

Opportunity is stuck.

33. RADAR SCORE

Example:

Radar Score =

ICP Fit

+

Contact Quality

+

Data Freshness

+

Engagement

+

Buying Signal

+

Historical Conversion Probability

Normalize to:

0–100

Categories:

90–100 🔥 Hot
75–89 High
50–74 Medium
25–49 Low
0–24 Cold

The scoring engine must be configurable.

34. JARVIS HOME SCREEN

Instead of a boring CRM dashboard:

Good Morning.

Jarvis Radar found:

🔥 12 high-value opportunities
⚠️ 7 follow-ups overdue
📈 4 leads moved closer to conversion
NEW 31 new matching companies
👤 8 decision-makers changed

Then:

[Start Working]

Opportunity #1
ABC Industries

Why Jarvis recommends this:

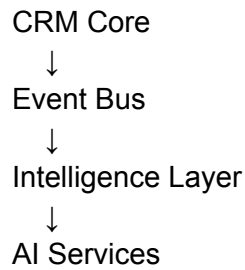
- Industry matches
- Decision-maker available
- Similar companies converted
- No contact in 14 days

Recommended action:
Call Procurement Head

35. AI LAYER

AI should not control core business logic initially.

Architecture:



Future AI capabilities:

- lead summarization
- company summarization
- contact summarization
- next best action
- email drafting
- WhatsApp drafting
- call summary
- conversation analysis
- lead scoring
- conversion prediction
- data cleansing
- duplicate detection
- enrichment
- campaign suggestions
- sales coaching

36. AI SAFETY PRINCIPLE

AI should NEVER bypass:

RBAC
Tenant Isolation
Field Permissions
Audit
Data Licensing

AI must operate within the same authorization boundary as the user.

37. MCP READINESS

Design tools as domain actions.

Examples:

search_global_companies
search_global_people
get_company
get_person
get_lead
create_lead
assign_lead
update_lead
create_task
create_followup
get_pipeline
get_radar
get_activity_history
create_note
send_email
prepare_whatsapp_message

These can later become MCP tools.

Do not tightly couple AI to database tables.

AI interacts with:

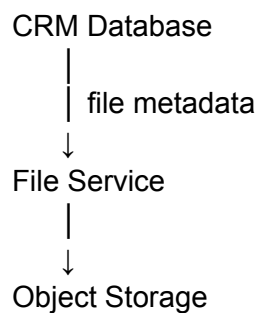
Domain Services

not raw SQL.

38. FILE MANAGEMENT ENGINE

Customer files must be isolated from CRM transactional database.

Recommended architecture:



Database stores:

file_id
organization_id
entity_type
entity_id
filename
mime_type
size
storage_provider
storage_key
checksum
uploaded_by
created_at

Actual file:

Object Storage

39. FILE STORAGE ABSTRACTION

Create:

StorageProvider

interface.

Example:

LocalStorageProvider
S3StorageProvider
CloudflareR2Provider
WasabiProvider
MinIOProvider

This allows future migration.

The application should not care where the physical file exists.

40. ONE-CLICK FILE MIGRATION

Migration service:

Export Metadata

+

Transfer Objects

+

Checksum Verification

+

Update Storage Keys

+

Switch Provider

Future API:

POST /storage/migrate

Migration must be resumable.

41. FILE SECURITY

Files must have:

- access control
- signed URLs
- MIME validation
- size limits
- malware scanning provision
- audit logs
- deletion policy

Never expose:

/uploads/customer/file.pdf

as a permanent public URL.

42. AUDIT ENGINE

Audit everything important.

Example:

USER_LOGIN
USER_LOGOUT
RECORD_VIEW
RECORD_CREATE

RECORD_UPDATE
RECORD_DELETE
RECORD_EXPORT
RECORD_IMPORT
LEAD_ASSIGN
LEAD_REASSIGN
PIPELINE_CHANGE
CALL
WHATSAPP
EMAIL
FILE_UPLOAD
FILE_DOWNLOAD
FILE_DELETE
PERMISSION_CHANGE
ROLE_CHANGE
DATA_PULL
GLOBAL_DATA_ACCESS

Audit record:

audit_id
organization_id
actor_id
action
entity_type
entity_id
timestamp
ip
user_agent
before
after
metadata

43. HIGH-VALUE DATA ACCESS AUDIT

For sensitive global/customer data:

Who
What
When
Why
From where
How many records
Which fields
What action

Example:

User: John

Action: Global Data Export

Records: 427

Time: 14:31

IP: xxx

Reason: Campaign

Approved by: Manager

For highly sensitive actions, support approval workflows.

44. EXPORT CONTROL

Export must never be treated as ordinary CRUD.

Permission:

export_global_data

export_customer_data

export_contacts

export_sensitive_fields

Potential controls:

No Export

Export Assigned Only

Export Team

Export Maximum 100

Export Requires Approval

Every export generates an audit event.

45. ADMIN DASHBOARD

Platform admin dashboard:

Organizations

Active Users

Global Companies

Global People

Global Relationships

Data Sources
New Imports
Failed Imports
Data Quality
Global Data Usage
Records Pulled
Exports
System Health
Storage Usage
API Usage
Subscription Revenue

46. CUSTOMER ADMIN DASHBOARD

Customer dashboard:

Total Leads
New Leads
Assigned
Unassigned
Contacted
Interested
Pipeline Value
Won
Lost
Conversion Rate
Overdue Followups
User Activity
Team Performance
Data Usage
Global Records Pulled

47. SALES USER DASHBOARD

Salesperson:

Today's Tasks
Today's Calls
Today's Followups
Hot Leads
Pending Leads
Overdue Leads
Meetings

48. TELECALLER DASHBOARD

Keep it extremely simple.

My Leads



Open Lead



Call



Outcome



Next Action

Possible outcomes:

Connected

No Answer

Busy

Wrong Number

Interested

Not Interested

Call Back

Meeting

This minimizes data exposure and maximizes productivity.

49. GLOBAL DATA RADAR

Paid users can browse:

Global Intelligence

Example:

Search:

Renewable Energy companies

Country:

Kenya

Employees:
50–500

Decision Maker:
Procurement

Email:
Available

Phone:
Available

Result:

128 Companies Found

[Select All]

Company A
Company B
Company C
...

Action:

[Add to CRM]

50. DATA CREDIT SYSTEM

Future monetization should support:

Monthly Subscription

+

Global Data Credits

+

Enrichment Credits

+

Export Credits

+

AI Credits

Example:

Starter

5,000 global records/month

Growth
25,000

Professional
100,000

Enterprise
Custom

Do not hard-code pricing into business logic.

Use:

plans
plan_features
usage_limits
usage_transactions
credits

51. SOURCE ATTRIBUTION

Every imported global record should know:

source_id
source_type
source_name
source_url
collection_date
license
confidence

This becomes important for:

- data quality
 - customer trust
 - legal compliance
 - source performance
 - commercial licensing
-

52. DATA COMPLIANCE

Jarvis should be designed for lawful data use.

The platform must support:

- source provenance
- data retention
- deletion requests
- correction requests
- suppression/Do-Not-Contact
- consent where required
- opt-out
- regional privacy requirements
- data processing records

Do not build the assumption that every scraped person's information can automatically be sold or used for outbound marketing.

The commercial/legal model for each dataset must be tracked.

53. DO-NOT-CONTACT ENGINE

Global and customer level suppression.

person_id
company_id
email
phone
domain
reason
source
created_at
expires_at

Before outbound:

Lead
↓
Suppression Check
↓
Allowed?
↓
Communication

54. NOTIFICATION ENGINE

Notification channels:

In-App
Email
Push
SMS
WhatsApp

Events:

Lead Assigned
Followup Due
Task Due
Lead Updated
Meeting Approaching
New Radar Opportunity
Import Complete
Import Failed
Approval Required

55. SEARCH ENGINE

Start with PostgreSQL full-text/trigram search.

Architecture should allow future:

OpenSearch / Elasticsearch

when scale requires it.

Do not introduce Elasticsearch unnecessarily in MVP.

56. RECOMMENDED TECH STACK

For the first production version:

Frontend

React
TypeScript

Vite
Tailwind CSS
shadcn/ui
React Router
TanStack Query
PWA

Backend

Recommended:

Python
FastAPI
Pydantic
SQLAlchemy
Alembic

Alternative:

Node.js
NestJS
TypeScript
Prisma

For Jarvis, I would choose:

FastAPI + PostgreSQL

because the future AI/data-processing layer will benefit from Python.

57. DATABASE

Primary:

PostgreSQL

Reasons:

- relational integrity
- JSONB
- full-text search
- indexing
- transactions
- mature ecosystem
- analytics

- multi-tenant architecture

Potential future:

Redis

OpenSearch

Vector DB

ClickHouse

Only introduce these when justified.

58. CACHE / QUEUE

Use:

Redis

for:

- caching
- background jobs
- rate limiting
- temporary state
- job queues

Background jobs:

Excel processing

CSV processing

deduplication

entity resolution

email

notifications

file processing

AI jobs

data enrichment

analytics

59. ARCHITECTURE STYLE

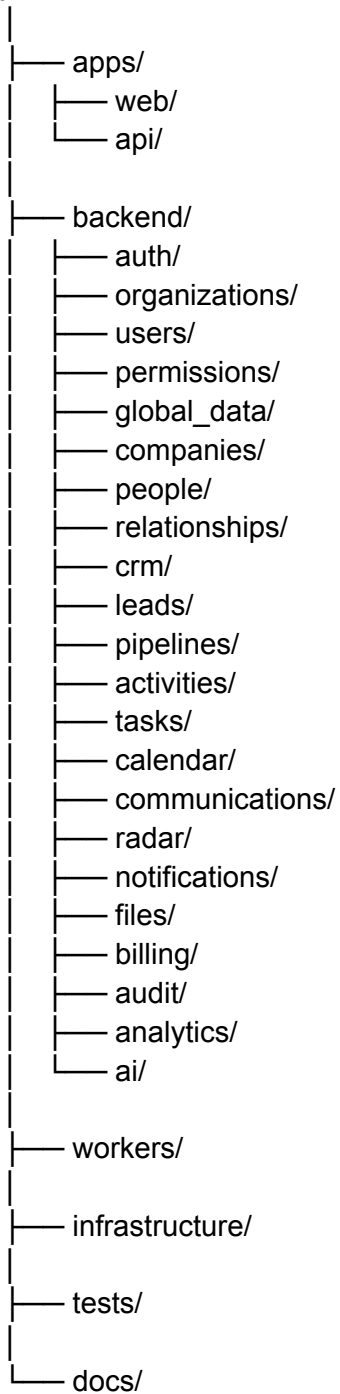
Do NOT start with microservices.

Use:

Modular Monolith

with clear domain boundaries.

jarvis/



60. DOMAIN BOUNDARIES

Each domain should have:

models
schemas
repository
service
routes
permissions
tests

Example:

```
leads/  
  models.py  
  schemas.py  
  repository.py  
  service.py  
  routes.py  
  permissions.py  
  tests/
```

Routes should not contain business logic.

61. SERVICE LAYER RULE

Bad:

API Route
↓
SQL

Good:

API
↓
Service
↓
Repository
↓
Database

AI:

AI Tool
↓
Domain Service
↓
Authorization

↓
Repository

This is essential for MCP readiness.

62. API DESIGN

REST API:

/api/v1/auth
/api/v1/organizations
/api/v1/users
/api/v1/roles
/api/v1/permissions

/api/v1/global/companies
/api/v1/global/people
/api/v1/global/imports

/api/v1/crm/companies
/api/v1/crm/people
/api/v1/leads
/api/v1/pipelines
/api/v1/activities
/api/v1/tasks
/api/v1/calendar

/api/v1/radar
/api/v1/communications
/api/v1/files
/api/v1/notifications
/api/v1/audit
/api/v1/analytics

63. API STANDARDS

Every endpoint must support:

authentication
authorization
tenant isolation
validation

pagination
sorting
filtering
audit where applicable
structured errors
request ID

Standard response:

```
{
  "success": true,
  "data": {},
  "meta": {
    "request_id": "..."
  }
}
```

Error:

```
{
  "success": false,
  "error": {
    "code": "LEAD_NOT_FOUND",
    "message": "Lead not found"
  },
  "meta": {
    "request_id": "..."
  }
}
```

64. PAGINATION

Never return thousands of records by default.

Default:

50 records

Maximum:

100–250 depending on endpoint

Use cursor pagination for high-volume datasets.

65. IMPORT ENGINE

Import must be asynchronous.

POST /imports



Create Import Job



Return Job ID



Background Worker



Process



Progress



Complete

UI:

Importing...



Records:

76,201 / 100,000

Duplicates:

3,204

Errors:

117

66. IMPORT ERROR MANAGEMENT

Never fail an entire 100,000-row import because of 20 invalid rows.

Produce:

Successful: 99,821

Duplicates: 159

Errors: 20

Allow:

Download Error Report
Fix
Re-upload

67. IMPORT PREVIEW

Before committing:

Detected:
Company Name → legal_name
CIN → CIN
Email → email
Mobile → mobile
Director Name → relationship

User confirms:

[Start Import]

68. DATA QUALITY SCORE

Each record can have:

Completeness
Accuracy
Freshness
Source Confidence
Relationship Confidence

Overall:

Data Quality Score: 87/100

69. DATABASE AUDIT

Database migrations must be version-controlled.

Use:

Alembic

Never manually modify production database structure.

Every schema change:

Migration

→ Test

→ Review

→ Deploy

70. SECURITY BASELINE

Mandatory:

HTTPS

JWT or secure session

HttpOnly cookies where appropriate

CSRF protection where applicable

password hashing

rate limiting

login throttling

RBAC

tenant isolation

input validation

SQL injection protection

XSS protection

secure headers

file validation

audit logs

secret management

71. AUTHENTICATION

MVP:

Email

Password

Session

Future:

Google

Microsoft

SSO

2FA

Passkeys

Architecture must permit extension.

72. SESSION SECURITY

Track:

session_id
user_id
created_at
last_activity
IP
user_agent
device
revoked_at

Allow:

Logout all devices
Revoke session

73. SUPER ADMIN SECURITY

Super Admin must have stronger controls.

Support:

2FA
IP restrictions
privileged action confirmation
audit
session monitoring

74. OBSERVABILITY

This is mandatory for AI vibe coding.

Every request gets:

request_id
trace_id
user_id
organization_id
endpoint
duration
status

Logs must be structured JSON.

Example:

```
{  
  "level": "INFO",  
  "event": "lead.created",  
  "request_id": "...",  
  "organization_id": "...",  
  "user_id": "...",  
  "lead_id": "..."  
}
```

75. ERROR TRACKING

Implement:

centralized error logging
structured logs
request tracing
background job monitoring
database error logging
frontend error boundary

Future integration can include:

Sentry
OpenTelemetry
Grafana
Prometheus

Do not hard-code one monitoring vendor.

76. DEBUG MODE

Development environment should provide:

DEBUG=true

with:

- SQL timing
- request ID
- API logs
- background job logs
- frontend errors
- import diagnostics

Production:

DEBUG=false

Never expose secrets or sensitive data in logs.

77. AI VIBE-CODING RULES

This project will be built with AI coding agents.

Therefore the repository must have explicit engineering rules.

Create:

AGENTS.md
CLAUDE.md
GEMINI.md
AI_RULES.md
ARCHITECTURE.md

These documents must explain:

- architecture
- naming
- coding standards
- security
- testing
- database rules
- tenant rules
- API conventions
- component conventions
- forbidden patterns

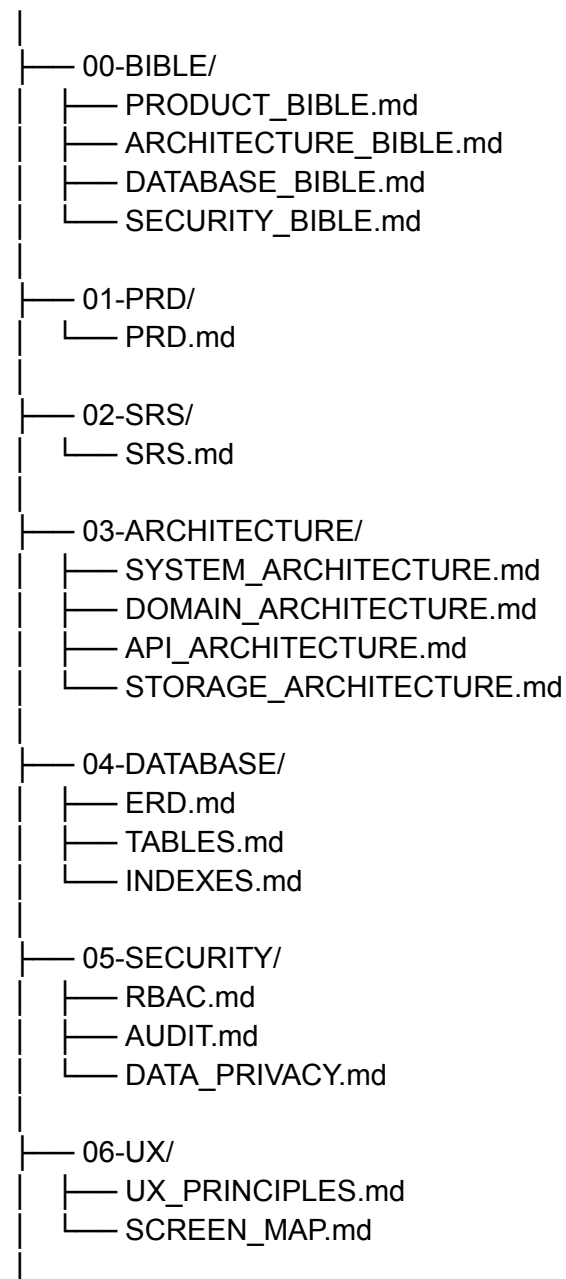
78. SOURCE OF TRUTH DOCUMENTATION

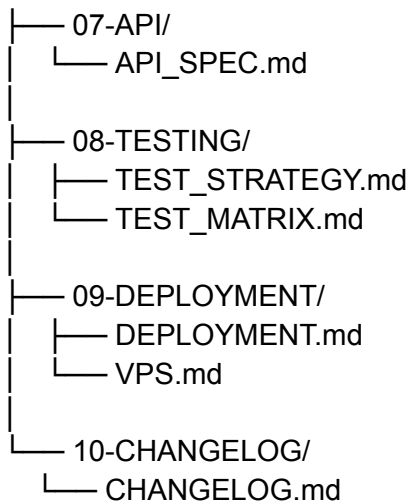
Create:

/docs

Structure:

docs/





79. PRODUCT BIBLE

The Product Bible is the highest-level source of truth.

It must define:

What Jarvis is

What Jarvis is not

Target users

Core problems

Product principles

Terminology

Business model

Core modules

Roadmap

Non-negotiable architecture rules

80. ARCHITECTURE BIBLE

Rules:

Rule 1

No cross-tenant data access.

Rule 2

No business logic inside UI components.

Rule 3

No business logic inside route controllers.

Rule 4

No direct database access from AI.

Rule 5

No hard-coded pipelines.

Rule 6

No hard-coded roles.

Rule 7

No hard-coded permissions.

Rule 8

No permanent local file paths.

Rule 9

No secrets in Git.

Rule 10

Every important mutation must be auditable.

81. TESTING STRATEGY

Four layers:

Unit Tests

Integration Tests

API Tests

End-to-End Tests

Critical tests:

Tenant Isolation

User A cannot access User B's records.

Role Security

Telecaller cannot export restricted data.

Global Data

Customer can only pull records allowed by plan.

Import

100,000 records can process asynchronously.

Duplicate Detection

Same company should not create unnecessary duplicates.

Audit

Sensitive actions generate audit events.

82. AUTOMATED TEST GATE

Every GitHub push:

Lint

↓

Type Check

↓

Unit Tests

↓

Integration Tests

↓

Build

↓

Security Checks

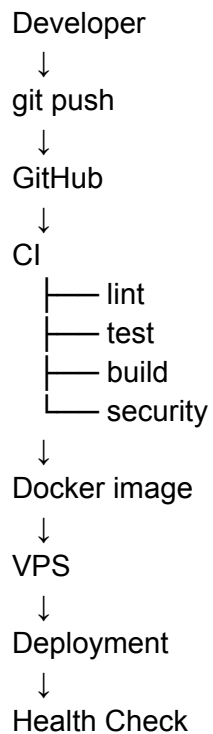
↓

Docker Build

Pull request cannot merge if required tests fail.

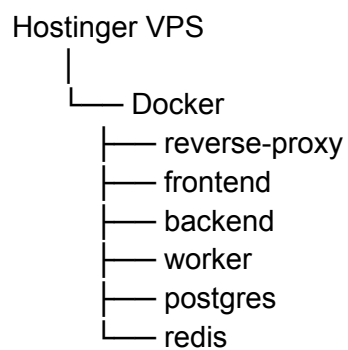
83. CI/CD

GitHub:



84. VPS DEPLOYMENT

Recommended:



Production files:

docker-compose.yml
docker-compose.prod.yml
.env
.env.example
Dockerfile

Never commit:

.env

85. REVERSE PROXY

Use:

Nginx

or equivalent.

Architecture:

Internet



HTTPS



Reverse Proxy

| — / → Frontend

| — /api → Backend

86. DATABASE BACKUP

Automated:

Daily PostgreSQL backup

+

Weekly full backup

+

Retention policy

Backup must be stored separately from the VPS.

Test restore regularly.

A backup that has never been restored is not a proven backup.

87. DEPLOYMENT SAFETY

Deployment process:

Backup
↓
Pull image
↓
Run migrations
↓
Start services
↓
Health check
↓
Smoke test
↓
Complete

If health check fails:

Rollback

88. HEALTH ENDPOINTS

Backend:

GET /health
GET /health/live
GET /health/ready

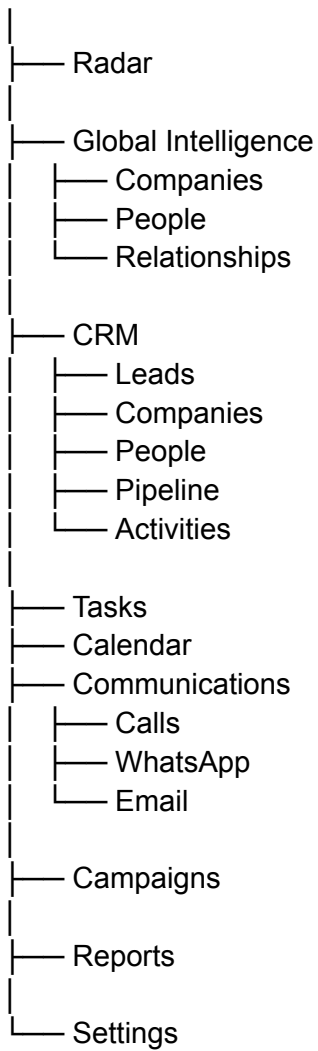
Check:

application
database
redis
worker
storage

89. FRONTEND INFORMATION ARCHITECTURE

Main navigation:

Jarvis
|
|— Home



Admin additionally:

Platform Admin



90. MOBILE/PWA UX

Mobile priority screens:

Home
Radar
My Leads
Lead Detail
Call
WhatsApp
Email
Followup
Tasks
Notifications

The lead detail screen should allow:

Call
WhatsApp
Email
Add Note
Add Activity
Change Stage
Schedule Followup

with minimal taps.

91. LEAD DETAIL PAGE

Example:

ABC Industries

 Radar Score: 87

Procurement Head
John Doe

 Call
 WhatsApp
 Email

Pipeline:
Interested

Next Follow-up:
Tomorrow 11:00

Company
Contacts
Timeline
Activities
Tasks
Documents
Notes
Signals

92. TIMELINE

The timeline becomes the single source for lead history.

Example:

Today 10:30
John called

Today 10:32
Lead moved:
Contacted → Interested

Today 10:35
Follow-up created

Yesterday 15:21
WhatsApp sent

Sep 2
Lead assigned to John

93. CAMPAIGN ENGINE

Future-ready campaign object:

campaign
audience
segment
sequence
steps
schedule
channel

owner
status

Example:

Campaign:
African Transformer Buyers

Audience:
Kenya + Tanzania

Sequence:

Email

↓

Wait 2 days

↓

Call

↓

WhatsApp

↓

Follow-up

94. SEGMENTATION

Users can create dynamic segments:

Industry = Energy

Country = Kenya

Employee > 50

Decision Maker = Procurement

Email Available = Yes

Lead Score > 70

Save as:

Kenya Energy Procurement Prospects

95. REPORTING

Reports:

Lead

- leads created
- leads assigned
- contacted
- connected
- interested
- won
- lost

User

- calls
- emails
- WhatsApp
- followups
- tasks
- conversion

Pipeline

- stage distribution
- stage duration
- conversion rate

Data

- global records pulled
 - imports
 - duplicates
 - exports
 - data quality
-

96. MANAGEMENT ANALYTICS

Management should see:

Activity ≠ Productivity

Calls:

1,000

Connected:

240

Interested:

63

Meetings:
28

Won:
7

This reveals the actual funnel.

97. AI FUTURE: SALES COPILOT

Jarvis should eventually say:

"You have 14 leads requiring attention today."

Then:

1. ABC Ltd
Last contacted 12 days ago
High probability
 2. XYZ Industries
Decision maker recently changed
Strong opportunity
 3. PQR Energy
Similar company converted last month
-

98. AI FUTURE: LEAD SUMMARY

ABC Industries

Company:
Industrial equipment manufacturer

Decision maker:
Procurement Director

History:
3 calls
2 emails
1 meeting

Current stage:
Proposal

Risk:
No activity for 8 days

Recommended:
Follow up with procurement.

99. AI FUTURE: NATURAL LANGUAGE CRM

Eventually:

"Show me all hot leads in Africa that haven't been contacted in 7 days."

Jarvis translates that into authorized domain operations.

Another:

"Assign the 20 highest priority Kenya leads to Rahul."

Jarvis:

Shows preview
→ confirms authorization
→ asks confirmation
→ performs action
→ records audit

100. AI ACTION SAFETY

Read actions:

Usually safe

Write actions:

Require stricter validation

High-risk actions:

Delete
Export
Mass message
Bulk assignment
Permission change

should require explicit confirmation.

101. DATABASE DESIGN PRINCIPLE

Do not prematurely create hundreds of tables.

Start with strong domains.

Core MVP tables:

organizations
users
roles
permissions
role_permissions

global_companies
global_people
global_company_people
global_sources
global_imports

crm_companies
crm_people
leads
lead_assignments
pipelines
pipeline_stages
pipeline_history

activities
calls
messages
emails
followups
tasks

notifications
audit_logs

files
file_links

subscriptions
plans
usage

Expand only when required.

102. SOFT DELETE

For important business records:

deleted_at
deleted_by

Avoid destructive deletion unless legally/business required.

103. RECORD VERSIONING

For sensitive records consider:

version
updated_by
updated_at

Critical entities can have history tables.

104. GLOBAL RECORD UPDATE STRATEGY

Suppose:

Global Company A

is already pulled by:

Customer X

Global database updates later.

Do NOT automatically overwrite the customer's CRM record.

Instead:

Global version changed



Customer copy may be stale



Show:

"Global data updated"



Customer decides:

[Review]

[Update]

[Ignore]

This is very important.

105. GLOBAL VS CUSTOMER DATA RELATION

Maintain:

global_record_id

customer_record_id

This creates lineage.

Example:

Global Company 123



Customer A Company 9001

Customer B Company 7712

Customer C Company 4411

106. DATA LINEAGE

Track:

Original Source

↓

Global Record

↓

Customer Import

↓

Lead

↓

Assignment

↓

Activity

↓

Conversion

This enables Jarvis to answer:

"Which data source produced the highest revenue?"

That becomes a major future product feature.

107. DATA SOURCE ROI

Example:

Source A

Records: 50,000

Pulled: 8,000

Contacted: 4,000

Interested: 500

Won: 80

Revenue: ₹40L

This lets Jarvis measure data-source quality.

108. GLOBAL DATA MONETIZATION

Potential commercial model:

Subscription

+
Data Credits
+
Premium Dataset Access
+
Enrichment
+
AI
+
Export

Dataset categories:

General Business
Industry
Country
Decision Makers
Verified Contacts
High Intent
Premium Intelligence

109. ADMIN DATA WORKFLOW

Platform team:

Upload
↓
Validate
↓
Normalize
↓
Deduplicate
↓
Enrich
↓
Approve
↓
Publish

Only:

PUBLISHED

data becomes available to customers.

110. DATA STATES

Global records:

DRAFT
PROCESSING
REVIEW
PUBLISHED
SUSPENDED
ARCHIVED

Import jobs:

UPLOADED
VALIDATING
PROCESSING
COMPLETED
PARTIAL
FAILED
CANCELLED

111. CUSTOMER LEAD STATES

Do not confuse pipeline stage with record status.

Example:

Lead Status:

ACTIVE
ON_HOLD
WON
LOST
ARCHIVED

Pipeline:

NEW
CONTACTED
INTERESTED
PROPOSAL
NEGOTIATION

112. CONFIGURATION ENGINE

Admin can configure:

- Lead fields
- Pipeline
- Stages
- Lead statuses
- Activity types
- Tags
- Custom fields
- User roles
- Permissions
- Notification rules
- Followup rules

Long-term:

Workflow Builder

113. CUSTOM FIELDS

Customers may add:

- Industry Segment
- Annual Turnover
- Product Interest
- Decision Timeline
- Budget
- Competitor
- Region

Do not modify database schema for every custom field.

Use a controlled custom-field framework.

114. WORKFLOW ENGINE — FUTURE

Example:

WHEN

Lead enters Proposal

IF

Lead value > ₹10 lakh

THEN
Create approval task

AND
Notify manager

AND
Schedule follow-up

This becomes the automation engine.

115. EVENT BUS

Create internal domain events.

Examples:

lead.created
lead.assigned
lead.stage_changed
activity.created
followup.due
global_record.pulled
import.completed
file.uploaded
user.created

AI and automation consume events.

116. INTERNAL EVENT ARCHITECTURE

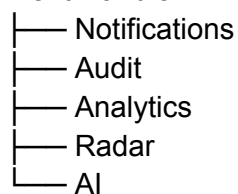
Domain Service



Event



Event Handler



This prevents tight coupling.

117. MVP DEFINITION

Do NOT attempt the entire vision in Version 1.

MVP should include:

Foundation

- authentication
- organizations
- users
- RBAC
- audit

Global Data

- company registry
- people registry
- relationship registry
- Excel/CSV upload
- mapping
- deduplication
- global search
- filters
- pull to CRM

CRM

- companies
- people
- leads
- assignment
- pipeline
- stages
- activities
- calls
- WhatsApp launch
- email
- notes
- followups
- tasks
- calendar

Dashboard

- admin dashboard
- user dashboard
- activity dashboard
- pipeline analytics

Infrastructure

- PWA
 - Docker
 - PostgreSQL
 - Redis
 - background jobs
 - CI/CD
 - backup
 - logging
 - tracing
-

118. MVP PHASE 2

Then:

Radar

Lead scoring

Data quality

Campaigns

Advanced segmentation

Notifications

File engine

Advanced analytics

119. PHASE 3

Then:

AI Copilot

AI summaries

Next best action

AI lead scoring

AI campaign generation

Conversation intelligence

120. PHASE 4

Then:

Global intelligence marketplace
Premium datasets
Automated enrichment
Intent signals
Predictive conversion
AI autonomous workflows

121. DEVELOPMENT ORDER

Build in this exact order.

Sprint 0 — Foundation

Repository
Docker
PostgreSQL
Redis
FastAPI
React
CI/CD
Documentation
Logging
Testing

Sprint 1 — Authentication

Users
Organizations
Sessions
RBAC

Sprint 2 — Global Data

Companies
People

Relationships
Sources

Sprint 3 — Import Engine

CSV
Excel
Mapping
Validation
Background jobs
Deduplication

Sprint 4 — Global Explorer

Search
Filter
Company detail
People detail
Relationship graph

Sprint 5 — Customer CRM

Companies
People
Leads
Assignments

Sprint 6 — Pipeline

Pipeline
Stages
History
Activities

Sprint 7 — Communications

Call
WhatsApp
Email
Followups

Sprint 8 — Dashboard

Admin
Manager
Sales
Telecaller

Sprint 9 — Audit + Security

Audit
Export control
Field security
Data access monitoring

Sprint 10 — Radar

Lead score
Signals
Recommendations

Sprint 11 — Files

File service
Object storage
Migration

Sprint 12 — AI Foundation

AI service
AI tools
MCP-ready domain tools

122. DEFINITION OF DONE

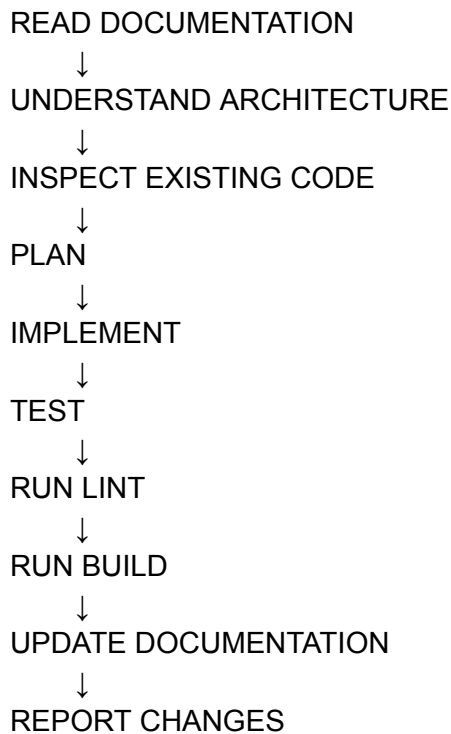
A feature is NOT complete merely because the UI works.

Every feature must have:

UI
API
Database
Authorization
Validation
Audit
Tests
Error handling
Loading state
Empty state
Responsive design
Documentation

123. AI CODING AGENT WORKFLOW

Every AI coding task must follow:



AI must never blindly rewrite an entire module.

124. AI AGENT RULE

Before changing code:

Do not assume.

Inspect first.

The AI coding agent should search:

existing components

existing services

existing models

existing migrations

existing API patterns

existing tests

before creating duplicates.

125. NO DUPLICATE CODE

Before creating:

service
component
utility
hook
API client
schema
permission

search repository first.

Reuse existing abstractions.

126. MIGRATION SAFETY

AI must NEVER:

drop production tables
truncate production data
delete migration history

without explicit human approval.

127. SEED DATA

Development environment should include:

demo organization
demo admin
demo manager
demo telecaller
demo sales user
sample companies
sample people
sample relationships
sample leads
sample activities

No real personal data.

128. TEST DATA GENERATOR

Create:

scripts/seed_demo.py

that can generate:

1,000 companies

5,000 people

10,000 relationships

5,000 leads

50,000 activities

This allows performance testing.

129. PERFORMANCE TARGETS

Initial targets:

Normal API

p95 < 500 ms

Search

p95 < 1 second

Dashboard

< 2 seconds where practical

Import

Must process large datasets asynchronously.

Never make browser wait for 100,000-row processing.

130. SCALABILITY PATH

MVP:

1 VPS
PostgreSQL
Redis
Docker

Growth:

Separate database server
Object storage
Read replicas
Worker scaling
Search engine

Enterprise:

Managed PostgreSQL
Redis cluster
Object storage
OpenSearch
Dedicated workers
CDN
Observability stack

Architecture should permit this migration.

131. API VERSIONING

Use:

/api/v1/

Future:

/api/v2/

Do not silently break existing APIs.

132. FEATURE FLAGS

Use feature flags for:

Radar

AI

Campaigns

MCP

Premium Data

Experimental features

Example:

feature_flags

This lets platform admins enable functionality selectively.

133. SUBSCRIPTION ENGINE

Tables:

plans

plan_features

subscriptions

subscription_events

usage

usage_transactions

Feature example:

global_search

global_pull

global_export

max_users

max_leads

ai_credits

storage_gb

134. BILLING ABSTRACTION

Create:

BillingProvider

so future providers can be added.

Do not couple the entire application to Stripe/Razorpay/etc.

135. PLATFORM ADMIN ROLES

Recommended:

SUPER_ADMIN
PLATFORM_ADMIN
DATA_ADMIN
DATA_RESEARCHER
SUPPORT_ADMIN
FINANCE_ADMIN
AUDITOR

Each gets controlled permissions.

136. GLOBAL DATA TEAM

Data researchers can:

upload
clean
review
verify
tag
publish

But should not automatically have:

billing
customer data
customer passwords
system configuration

137. SUPPORT TEAM

Support should use:

impersonation / support session

only with strict controls.

Every support access:

audited
time limited
reason required

138. CUSTOMER ADMIN DATA CONTROL

Customer admin controls:

users
roles
teams
lead assignment
pipeline
custom fields
campaigns
data access
exports
integrations

But cannot access:

other organizations
platform internal data
restricted global source information

unless authorized.

139. DATABASE INDEX STRATEGY

Indexes should exist for common access patterns:

tenant_id
tenant_id + created_at
tenant_id + owner_id
tenant_id + status
tenant_id + stage_id
tenant_id + phone
tenant_id + email

Global:

CIN
DIN
domain
email
phone
normalized_company_name
country
city
industry

Index based on actual query patterns.

140. DATA RETENTION

Configurable:

activity retention
audit retention
deleted records
file retention
notification retention
import history

Do not hard-code retention periods.

141. GDPR/PRIVACY-STYLE OPERATIONS

Even if initial target markets are outside Europe, design:

export personal data
delete personal data
correct data
suppress contact
record lawful basis where needed
track data source

This will make international expansion easier.

142. OBSERVABILITY DASHBOARD

Admin should eventually see:

API health
Error rate
Latency
Queue depth
Failed jobs
Database health
Storage health
Active users
Login failures
Import failures

143. SYSTEM DIAGNOSTICS

Create:

/admin/system/health

with:

Database: ✓
Redis: ✓
Worker: ✓
Storage: ✓
Email: ✓
WhatsApp: configured
AI: configured

144. CHANGELOG

Every release:

Added
Changed
Fixed
Security
Database

145. ARCHITECTURE DECISION RECORDS

Use:

docs/architecture/ADR-001.md
ADR-002.md
...

Examples:

ADR-001 Modular Monolith
ADR-002 PostgreSQL
ADR-003 Global/Customer Data Separation
ADR-004 Object Storage Abstraction
ADR-005 RBAC Model
ADR-006 Event Architecture
ADR-007 AI/MCP Boundary

This prevents AI coding agents from repeatedly questioning/reversing architecture.

146. UI DESIGN PRINCIPLE

Jarvis should feel:

Intelligent
Fast
Professional
Data-rich
Modern
Operational

Avoid:

generic CRM dashboard
too many cards
unnecessary gradients
huge tables
cluttered navigation

147. JARVIS VISUAL IDENTITY

The interface should visually communicate:

Radar
Intelligence
Signals
Action

Not:

Accounting software

Use a clean enterprise SaaS design.

148. GLOBAL SEARCH

Global search should eventually search:

Companies
People
Leads
Activities
Tasks

Example:

Search:
"ABC"

Results:

Companies
ABC Industries

People
Amit ABC

Leads
ABC Industries Opportunity

Activities

149. COMMAND CENTER

Future:

Ctrl + K

opens:

Jarvis Command Center

Commands:

Search company

Create lead

Assign lead

Schedule followup

Open radar

Search global database

Create task

Later powered by AI.

150. PRODUCT NORTH STAR

Jarvis should ultimately answer three questions for management:

1.

Where are my opportunities?

→ Radar

2.

What is my team doing?

→ Activity Intelligence

3.

What should we do next?

→ Jarvis AI

151. NORTH-STAR METRIC

Primary:

Qualified Opportunities Converted Per Unit of Data

Secondary:

Global data → CRM pull rate

CRM → contacted rate

contacted → interested rate

interested → opportunity rate

opportunity → won rate

152. THE BIG DIFFERENTIATOR

Traditional CRM:

Database of known leads

+

Activity tracking

Jarvis:

Global Intelligence

+

Controlled Data Access

+

CRM

+

Activity Intelligence

+

Lead Radar

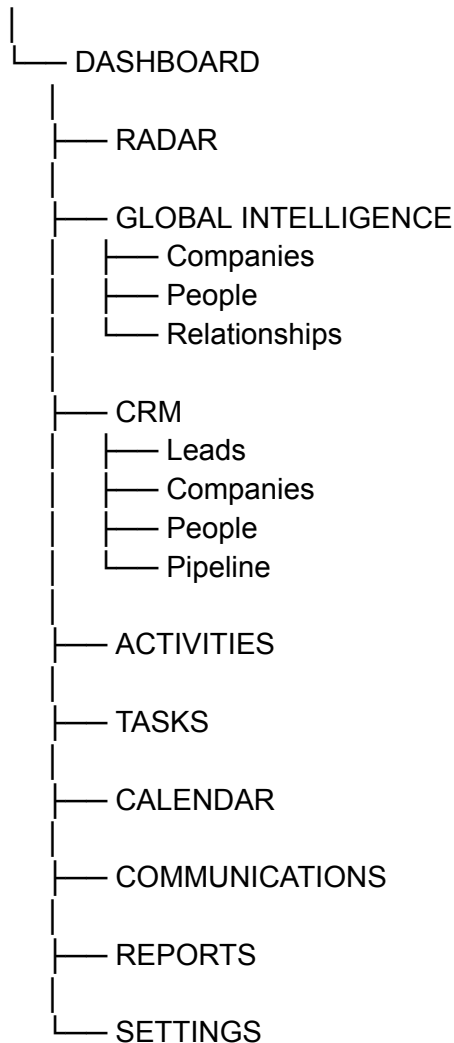
+

AI

That is the product moat.

153. INITIAL SCREEN MAP

LOGIN



154. PLATFORM ADMIN SCREEN MAP

ADMIN



—	Data Usage
—	Subscriptions
—	Plans
—	Credits
—	Audit
—	System Health
—	Jobs
—	Storage
—	Platform Settings

155. ACCEPTANCE CRITERIA — MVP

The MVP is successful when:

Global Data

A platform admin can upload a 100,000-row Excel file.

Jarvis:

detects columns
maps fields
validates
deduplicates
imports
tracks progress
reports errors

Customer

A paid organization can:

search global database
select records
pull records
see them in CRM
assign them

Employee

Employee can:

open assigned lead
call
WhatsApp

email
record outcome
schedule followup
change stage

Management

Admin can see:

activity
pipeline
conversion
team performance

Security

Employee cannot:

access unauthorized records
export unauthorized data
see restricted fields
access another organization

Audit

Management can determine:

who viewed
who changed
who assigned
who exported
who contacted

156. AI DEVELOPMENT MASTER PROMPT

Use the following as the system prompt for an AI coding agent:

JARVIS ENGINEERING AGENT

You are the Senior Software Architect and Principal Engineer for Jarvis CRM.

Jarvis is a multi-tenant SaaS platform combining:

1. Global Business Intelligence Database
2. Company Registry
3. People Registry
4. Relationship Graph
5. Controlled Customer CRM
6. Lead Pipeline
7. Activity Tracking
8. Communication Tracking
9. Lead Intelligence Radar
10. AI/MCP-ready intelligence layer

You MUST follow [/docs/](#) as the source of truth.

Before changing code:

1. Inspect existing implementation.
2. Inspect architecture documentation.
3. Search for existing reusable components/services.
4. Understand tenant boundaries.
5. Understand authorization.
6. Create an implementation plan.
7. Implement the smallest correct change.
8. Write/update tests.
9. Run lint/type checks/tests.
10. Update documentation if architecture changed.

NEVER:

- bypass tenant isolation
- access another organization's records
- put business logic in UI
- put business logic in route handlers
- access DB directly from AI
- hard-code roles
- hard-code permissions
- hard-code pipelines
- expose secrets
- log sensitive personal data unnecessarily
- create duplicate services/components
- delete migrations
- modify production destructively
- introduce unnecessary dependencies

Every important mutation must be auditable.

Every API must validate authorization.

Every customer-owned record must be tenant scoped.

Every file must use the storage abstraction.

Every AI capability must use domain services.

When uncertain, STOP and explain the architectural uncertainty rather than inventing an implementation.

157. AI CODING TASK FORMAT

Every coding request should be structured as:

TASK ID:

TITLE:

OBJECTIVE:

BUSINESS REQUIREMENT:

FILES TO INSPECT:

EXPECTED BEHAVIOR:

DATABASE CHANGES:

API CHANGES:

UI CHANGES:

PERMISSION REQUIREMENTS:

AUDIT REQUIREMENTS:

TEST REQUIREMENTS:

ACCEPTANCE CRITERIA:

DO NOT CHANGE:

158. EXAMPLE AI TASK

TASK ID:
CRM-LEAD-001

TITLE:
Create Lead Assignment Engine

OBJECTIVE:
Allow managers to assign CRM leads to authorized users.

RULES:

- Manager can assign only leads belonging to their organization.
- Manager can assign only to users within their allowed scope.
- Telecaller cannot reassign leads.
- Every assignment creates an audit event.
- Assignment history must be preserved.

TESTS:

- authorized assignment succeeds
- cross-tenant assignment fails
- unauthorized user fails
- audit event created

159. DEVELOPMENT COMMANDS

Standardize:

make install
make dev
make test
make lint
make format
make typecheck
make migrate
make seed
make build
make docker
make health

One command should start local development:

make dev

160. ENVIRONMENT MANAGEMENT

Use:

.env.example
.env.local
.env.production

Variables:

DATABASE_URL
REDIS_URL
JWT_SECRET
SESSION_SECRET
STORAGE_PROVIDER
STORAGE_BUCKET
STORAGE_ACCESS_KEY
STORAGE_SECRET_KEY
EMAIL_PROVIDER
WHATSAPP_PROVIDER
AI_PROVIDER
SENTRY_DSN

Secrets must never be committed.

161. GITHUB BRANCHING

Recommended:

main
develop
feature/*
fix/*
hotfix/*

Production:

main

Pull request:

feature
↓
CI
↓
Review
↓
Merge

↓
Deploy

162. RELEASE STRATEGY

Version:

v0.1.0
v0.2.0
v0.3.0
v1.0.0

Before v1:

security audit
tenant isolation test
backup restore test
load test
UAT

163. MVP PRIORITY MATRIX

P0 — Must Have

Multi-tenancy
Authentication
RBAC
Company Registry
People Registry
Relationships
Import
Global Search
Global → CRM Pull
Leads
Assignments
Pipeline
Activities
Followups
Tasks
Audit
PWA
Docker
CI/CD

Backup

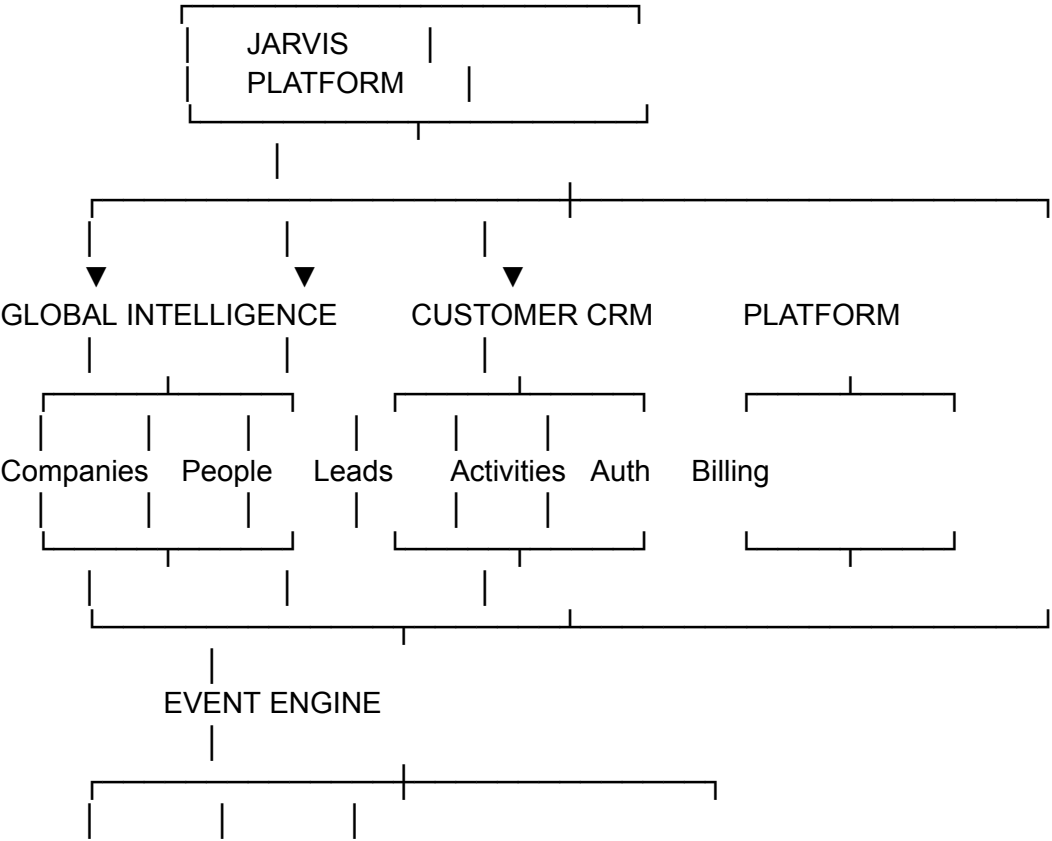
P1

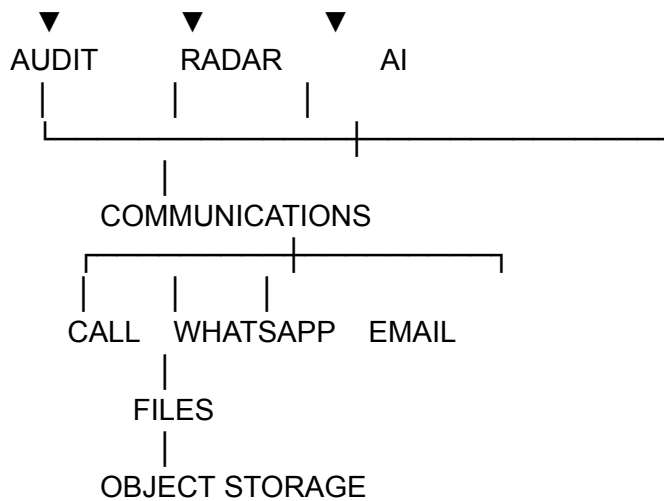
- Radar
- Advanced Analytics
- Campaigns
- File Engine
- Notifications
- Data Quality
- Credits

P2

- AI Copilot
- MCP
- Enrichment
- Predictive Scoring
- Intent Detection
- Autonomous Workflows

164. FINAL ARCHITECTURE





165. FINAL PRODUCT PRINCIPLE

Jarvis should not become:

"Another CRM with AI added to it."

It should become:

"An intelligence operating system for outbound sales."

The fundamental loop is:

DISCOVER



UNDERSTAND



SELECT



CONTROL



ASSIGN



CONTACT



TRACK



LEARN



PRIORITIZE



CONVERT

And eventually:

Jarvis discovers the opportunity



Jarvis explains why it matters



Jarvis recommends who should contact it



Human executes



Jarvis records the result



Jarvis learns from the result



Jarvis finds the next opportunity

That is the core product philosophy of Jarvis CRM.

My architectural recommendation

I would **not build this as a huge microservice system initially**. For your AI/vibe-coding approach, the sweet spot is:

React + TypeScript + PWA → FastAPI + Python → PostgreSQL + Redis → background workers → object storage → Docker → Hostinger VPS → GitHub CI/CD.

And I would make **Global Intelligence, Customer CRM, Audit/Security and File Storage separate architectural domains from Day 1**, even though they initially run in one modular monolith.